

LINQ Operators in C#

In this article, I will discuss the **LINQ Operators in C#**. Please read our previous article discussing the [LINQ Extension Methods](#) and how they are implemented. As part of this article, we will discuss the following two points.

1. **What are LINQ Operators?**
2. **What are the Categories of LINQ Operators?**

What are LINQ Operators?

The LINQ Operators are nothing but a set of extension methods used to write the LINQ Query. These LINQ extension methods provide many useful features we can apply to the data source. Some of the features are filtering the data, sorting the data, grouping the data, etc.

What are the Categories of LINQ Operators?

LINQ (Language-Integrated Query) operators in C# provide a way to query and manipulate data from arrays, enumerable classes, XML, relational databases, and third-party data sources. The operators are divided into different categories based on their functionality:

Projection Operators:

These operators transform the elements of a sequence into a new form. Common projection operators include Select and SelectMany.

- **Select:** Projects each element of a sequence into a new form.
- **SelectMany:** Projects each sequence element to an `IEnumerable<T>` and flattens the resulting sequences into one sequence.

Filtering Operators:

These are used for filtering data. The most common restriction operator is Where which applies a predicate to each element of a sequence and returns those that satisfy the condition.

- **Where:** Filters a sequence of values based on a predicate.
- **OfType:** Filters the elements of an array based on a specified type.

Partitioning Operators:

These operators divide a sequence into two parts and return one of them. Examples include Take, Skip, TakeWhile, and SkipWhile.

- **Take:** Returns a specified number of contiguous elements from the start of a sequence.
- **Skip:** Bypasses a specified number of elements in a sequence and then returns the remaining elements.

- **TakeWhile:** Returns elements from a sequence as long as a specified condition is true.
- **SkipWhile:** Bypasses elements in a sequence as long as a specified condition is true and then returns the remaining elements.

Ordering Operators:

These operators arrange the elements of a sequence. Common ordering operators are `OrderBy`, `OrderByDescending`, `ThenBy`, and `ThenByDescending`.

- **OrderBy:** Sorts the elements of a sequence in ascending order according to a key.
- **OrderByDescending:** Sorts the elements of a sequence in descending order according to a key.
- **ThenBy:** Performs a subsequent ordering of the elements in a sequence in ascending order.
- **ThenByDescending:** Performs a subsequent ordering of the elements in a sequence in descending order.
- **Reverse:** Inverts the order of the elements in a sequence.

Grouping Operators:

These operators group elements of a sequence based on a specified key value. The most notable grouping operator is `GroupBy`.

- **GroupBy:** Groups the elements of a sequence according to a specified key selector function.

Join Operators:

These operators are used to combine elements from two or more sequences. Common join operators are `Join` and `GroupJoin`.

- **Join:** Joins two sequences based on matching keys.
- **GroupJoin:** Groups elements from a sequence based on a key and joins them with elements from another sequence.

Set Operators:

These operators perform mathematical set operations on sequences, such as `Distinct`, `Union`, `Intersect`, and `Except`.

- **Distinct:** Removes duplicate elements from a sequence.
- **Union:** Produces the set union of two sequences.
- **Intersect:** Produces the set intersection of two sequences.
- **Except:** Produces the set difference of two sequences.

Conversion Operators:

These are used to convert one type of sequence or collection to another. Examples include `ToArray`, `ToList`, `ToDictionary`, and `AsEnumerable`.

- **AsEnumerable:** Casts an `IEnumerable` to an `IEnumerable<T>`.

- **ToArray:** Converts a sequence to an array.
- **ToList:** Converts a sequence to a List<T>.
- **ToDictionary:** Converts a sequence to a Dictionary<TKey, TValue> based on a key selector function.

Element Operators:

These operators return a single element from a sequence. Examples include First, FirstOrDefault, Last, LastOrDefault, Single, SingleOrDefault, and ElementAt.

- **First:** Returns the first element of a sequence.
- **FirstOrDefault:** Returns the first element of a sequence or a default value if no element is found.
- **Last:** Returns the last element of a sequence.
- **LastOrDefault:** Returns the last element of a sequence or a default value if no element is found.
- **Single:** Returns the only element of a sequence and throws an exception if there is not exactly one element in the sequence.
- **SingleOrDefault:** Returns the only element of a sequence or a default value if the sequence is empty; this method throws an exception if there is more than one element in the sequence.
- **ElementAt:** Returns the element at a specified index in a sequence.
- **ElementAtOrDefault:** Returns the element at a specified index in a sequence or a default value if the index is out of range.

Quantifier Operators:

These operators return a Boolean value indicating whether all or any of the elements of a sequence satisfy a condition. Examples are All, Any, and Contains.

- **Any:** Determines whether any element of a sequence satisfies a condition.
- **All:** Determines whether all elements of a sequence satisfy a condition.
- **Contains:** Determines whether a sequence contains a specified element.

Aggregate Operators:

These operators perform a calculation on a sequence and return a single value. Examples include Count, Sum, Min, Max, Average, and Aggregate.

- **Count:** Counts the elements in a sequence.
- **LongCount:** Counts the elements in a sequence, returning the count as a long.
- **Sum:** Computes the sum of a sequence of numeric values.
- **Min:** Returns the minimum value in a sequence.
- **Max:** Returns the maximum value in a sequence.
- **Average:** Computes the average of a sequence of numeric values.
- **Aggregate:** Applies an accumulator function over a sequence.

Equality Operators:

These operators are used to compare sequences for equality. An example is SequenceEqual.

- **SequenceEqual:** Determines whether two sequences are equal by comparing the elements by using the default equality comparer for their type.

Generation Operators:

These operators are used to create a new sequence of values. Examples include Range, Repeat, and Empty.

- **Empty:** Returns an empty IEnumerable<T> with the specified type argument.
- **Repeat:** Generates a sequence that contains one repeated value.
- **Range:** Generates a sequence of integral numbers within a specified range.

Special Operators:

- **Concatenation Operators:** These operators concatenate two sequences. The primary operator in this category is Concat.
- **DefaultIfEmpty Operators:** This operator returns the elements of the specified sequence or the type parameter's default value in a singleton collection if the sequence is empty.

These operators provide powerful capabilities for querying and manipulating collections in C#. They can be used with various types of data sources, including arrays, lists, XML, and even databases through LINQ to SQL or Entity Framework. In the next article, I will discuss [Projection Operations](#) with examples.